Before you turn this problem in, make sure everything runs as expected. First, **restart the kernel** (in the menubar, select Kernel→Restart) and then **run all cells** (in the menubar, select Cell→Run All).

Make sure you fill in any place that says `YOUR CODE HERE` or "YOUR ANSWER HERE", as well as your name and collaborators below:

In [1]:
```
NAME = "AMEY GAIKWAD"
COLLABORATORS = "SVEN HARDER"
```

Physics C170M/270M Lab 2: Naive Bayes

In this lab, you will

- Incorporate the 8 steps of machine learning
- Calculate a small Naive Bayes problem by hand
- Code a Gaussian Naive Bayes model to analyze the iris dataset.
- Use scikit-learn and Naive Bayes models to make predictions
- Application of Naive Bayes in Astrophysics: classify stars at the center of our Galaxy

Reminder: save and checkpoint often!

Lab Created by: Tuan Do & Bernie Boscoe

Last updated by: Tuan Do

Naive Bayes exercise by hand, with machine learning language

Bayes theorem: $P(A \mid B) = \frac{P(A,B)}{P(B)} = \frac{P(B|A) \times P(A)}{P(B)}$ Basically, we are trying to find probability of event A, given the event B is true. Event B is also termed as evidence. P(A) is the prior of A (the prior probability, i.e. Probability of event before evidence is seen). The evidence is an attribute value of an unknown instance(here, it is event B). P(A|B) is a posteriori probability of B, i.e. probability of event after evidence is seen.

Let $(x_1, x_2, \ldots, x_p)$ be a feature vector and y be the class label corresponding to this feature vector. Applying Bayes' theorem,

$$P(y \mid X) = \frac{P(X, y)}{P(X)} = \frac{P(X \mid y) \times P(y)}{P(X)}$$

where X is given as $X = (x_1, x_2, \ldots, x_p)$. By substituting for X and expanding using the chain rule we get,

$$P(y \mid x_1, x_2, \ldots, x_p) = \frac{P(x_1, x_2, \ldots, x_p, y)}{P(x_1, x_2, \ldots, x_p)} = \frac{P(x_1, x_2, \ldots, x_p \mid y) \times P(y)}{P(x_1, x_2, \ldots, x_p)}$$

Since $(x_1, x_2, \ldots, x_p)$ are independent of each other,

$$P(y \mid x_1, x_2, \ldots, x_p) = \frac{P(y) \times \prod_{i=1}^{p} P(x_i \mid y)}{\prod_{i=1}^{p} P(x_i)}$$

For all entries in the dataset, the denominator does not change, it remains constant. Therefore, the denominator can be removed and a proportionality can be introduced:

$$P(y \mid x_1, x_2, \ldots, x_p) \propto P(y) \times \prod_{i=1}^{p} P(x_i \mid y)$$

In our scikit learn exercise, the response variable (y) has only two outcomes, binary (e.g., yes or no / positive or negative). There could be cases where the classification could be multivariate.

To complete the specification of our classifier, we adopt the MAP (Maximum A Posteriori) decision rule, which assigns the label to the class with the highest posterior.

$$\hat{y} = p(X, y) = p(y, x_1, x_2, \ldots, x_p) = \underset{k \in \{1,2,\ldots,K\}}{\operatorname{argmax}} \; P(y) \times \prod_{i=1}^{p} P(x_i \mid y)$$

We calculate probability for all ‘K’ classes using the above function and take one with the maximum value to classify a new point belongs to that class.

Let's look at a naive bayes by hand example, using data science jargon. The data we will use is various aspects of weather conditions and a binary outcome, whether or not one plays golf.

The dataset is divided into two parts, namely, feature matrix and the response vector.

The feature matrix contains all the vectors(rows) of dataset in which each vector consists of the value of dependent features. In our dataset, features are ‘Temperature’, ‘Humidity’ and ‘Weather’. Response vector contains the value of class variable(prediction or output) for each row of feature matrix. In our dataset, the class variable name is ‘Play golf’. Here is a list of the categorical features and the values they take, as well as the binary predicted value.

Temperature	Weather	Humidity	Play Golf
Hot	Sunny	High	Yes

| Cold | Rainy | Low |No

Let's look at ten events and outcomes:

Temperature	Weather	Humidity	Play Golf
Hot	Sunny	High	Yes
Hot	Rainy	High	Yes
Hot	Rainy	High	Yes
Cold	Sunny	Low	No
Hot	Sunny	Low	No
Cold	Rainy	Low	No
Cold	Sunny	High	Yes
Cold	Rainy	Low	No
Cold	Sunny	High	No
Cold	Rainy	Low	Yes

Just to clear, an example of a feature vector and corresponding class variable can be: (refer to 1st row of dataset)

X = (Hot, Sunny , High)

y = Yes

So basically, P(y|X) here means, the probability of “playing golf” given that the weather conditions are “Temperature is hot”, "sunny", and “high humidity”.

Naive assumption

Now, its time to put a naive assumption to the Bayes’ theorem, which is, independence among the features. So now, we split evidence into the independent parts.

Now, if any two events A and B are independent, then,

P(A,B) = P(A)P(B)

So, finally, we are left with the task of calculating $P(y)$ and $P(x_i|y)$.

Please note that $P(y)$ is also called the class probability and $P(x_i|y)$ is called the conditional probability.

The different naive Bayes classifiers differ mainly by the assumptions they make regarding the distribution of $P(x_i|y)$.

Let us try to apply the above formula manually on our weather dataset. For this, we need to do some precomputations on our dataset.

We need to find $P(x_i|y_j)$ for each x_i in X and y_j in y.

Fill in the missing values, by computing the priors. (Just use a piece of paper, you will need them for a later question)

Temperature

|| Yes | No | P(yes)|P(no) | ---------- |----------| ------|--------|----| | Hot | 3| 1 |3/5|1/5| | Cold | 2 | ?|P(Cold | yes)=?|P(Cold | no)=?| | Total |5| 5 |100%|100%|

Weather

|| Yes | No | P(yes)|P(no) | ---------- |----------| ------|--------|----| | Sunny | 2| 3 |2/5|3/5| | Rainy | ? | ?|P(rainy | yes)=?|P(Rainy | no)=?| | Total |5| 5 |100%|100%|

Humidity

|| Yes | No | P(yes)|P(no) | ---------- |----------| ------|--------|----| | High | 4| 1 |4/5|1/5| | Low | ? | ?|?|?| | Total |5| 5 |100%|100%|

Filled out missing values

Temperature

|| Yes | No | P(yes)|P(no) | ---------- |----------| ------|--------|----| | Hot | 3| 1 |3/5|1/5| | Cold | 2 | ?|P(Cold | yes)=2/5 |P(Cold | no)=4/5| | Total |5| 5 |100%|100%|

Weather

|| Yes | No | P(yes)|P(no) | ---------- |----------| ------|--------|----| | Sunny | 2| 3 |2/5|3/5| | Rainy | ? | ?|P(rainy | yes)=3/5|P(Rainy | no)=2/5| | Total |5| 5 |100%|100%|

Humidity

|| Yes | No | P(yes)|P(no) | ---------- |----------| ------|--------|----| | High | 4| 1 |4/5|1/5| | Low | 1 | 4|1/5|4/5| | Total |5| 5 |100%|100%|

Question 1

(2 pts.) So, in the figures above, we have calculated $P(x_i|y_j)$ for each x_i in X and y_j in y manually in the tables 1-4. For example, probability of playing golf given that the temperature is cool, i.e P(temp. = cold | play golf = Yes) = 2/5.

Also, we need to find class probabilities $(P(y))$. For example, P(play golf = Yes) = 5/10.

Play Golf

| Play | P(yes)|P(no) | ---------- | ------|--------| | Yes | 5|?| |No | 5|5/10| | Total |10| 100%|

today = (Sunny, Hot, Low)

so $P(Yes|today)$ = $P(Sunny|Yes) * P(Hot|Yes) * P(Low|Yes) * P(Yes)$ (we can ignore denominator) = ?

Type in the four fractions below, and the result when you multiply them together.

Filled out table :

Play Golf

| Play | P(yes)|P(no) | ---------- | ------|--------| | Yes | 5|5/10| |No | 5|5/10| | Total |10| 100%|

$P(Sunny|Yes) = 2/5$

$P(Hot|Yes) = 3/5$

$P(Low|Yes) = 1/5$

$P(Yes) = 5/10$

Hence,

$$ P(Yes|today) = 2/5 \times 3/5 \times 1/5 \times 5/10 = 3/125

Question 2

(1 pt.) Now, what is P(No | Today)? Next, convert these two values into probabilities by making the sum equal to 1 (normalization): example: P(Yes|today)/ P(Yes |today) + P(No | today) **Write the probability that golf will be played today. How likely is it golf will be played today?**

Unnormalised P(No|Today):

$P(Sunny|No) = 3/5$ $P(Hot|No) = 1/5$ $P(Low|No) = 4/5$ $P(No) = 5/10$

Hence,

$$P(No|Today) = 3/5 \times 1/5 \times 4/5 \times 5/10 = 6/125$$

Normalise :

$$P(Yes|today) = 3/9 = 1/3$$

$$P(No|today) = 3/9 = 2/3$$

Iris Classification with Naive Bayes

In this part of the assignment, you will again use the Iris dataset. Recall, it consists of 50 samples from each of three species of Iris (Iris setosa, Iris virginica and Iris versicolor). Four features were measured from each sample: the length and the width of the sepals and petals, in centimeters. For a reference, see the following papers:

- R. A. Fisher. "The use of multiple measurements in taxonomic problems". Annals of Eugenics. 7 (2): 179–188, 1936.

Your goal is to construct a Naive Bayes classifier model that predicts the correct class from the sepal length and sepal width features. This lab will help learn about using probability distributions in the Naive Bayes classifier.

In [2]:
```
# run this cell
#### PACKAGE IMPORTS ####
import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn.metrics import accuracy_score
from sklearn import datasets, model_selection
import pandas as pd
import seaborn as sns
```

Step 1: Look at the big picture

What is the problem you want to solve? How do you plan to use and benefit from the the machine learning model? What is your metric for success?

In this exercise, we will give you the goal and the metric:

Machine Learning Goal: classify irises based on the measurement of their petals.

Metric: accuracy - defined as the number of correctly typed irises divided by the total number of irises

Step 2: Get the data

What data are available to you for training your model? How will you access and download all the data? The type and quantity of data available will often impact your choice in the subsequent stages.

Scikit Learn has a bunch of pre-built machine learning datasets that are helpful for learning new models and testing.

Load and prepare the data

We will first read in the Iris dataset, and then split the dataset into training and test sets. The extra cell below the code solution is provided to check your work.

Question 3

In [3]:
```
# (1 pt.) Load the iris dataset (hint: see how Lab 1 loaded the Iris dataset
# YOUR CODE HERE
iris=datasets.load_iris()
```

Step 3: Explore the data

Discover and visualize the data to gain insight. For example, are there more representatives of certain types of data than others?

Question 4

(6 pts)

For this lab, please answer the following questions about your data. Use as many cells and plots as you like.

1. What are the features? What is the label?
2. Compute some summary statistics about your dataset. For example, the number of samples, mean, median, dispersion of the features.
3. Plot some of the features to get a sense of the data.

1. Features -- sepal length, sepal width, petal length, petal width ; Label -- iris species (setosa, versicolor, virginica)

In [4]:
```
## 2. Summary statistics
df=pd.DataFrame(iris.data, columns=iris.feature_names)
df.describe()
## Example of median
df.median()
```

Out[4]:
```
sepal length (cm)    5.80
sepal width (cm)     3.00
petal length (cm)    4.35
petal width (cm)     1.30
dtype: float64
```

In [5]:
```
## 3. Plotting some features
X= iris.data
y=iris.target

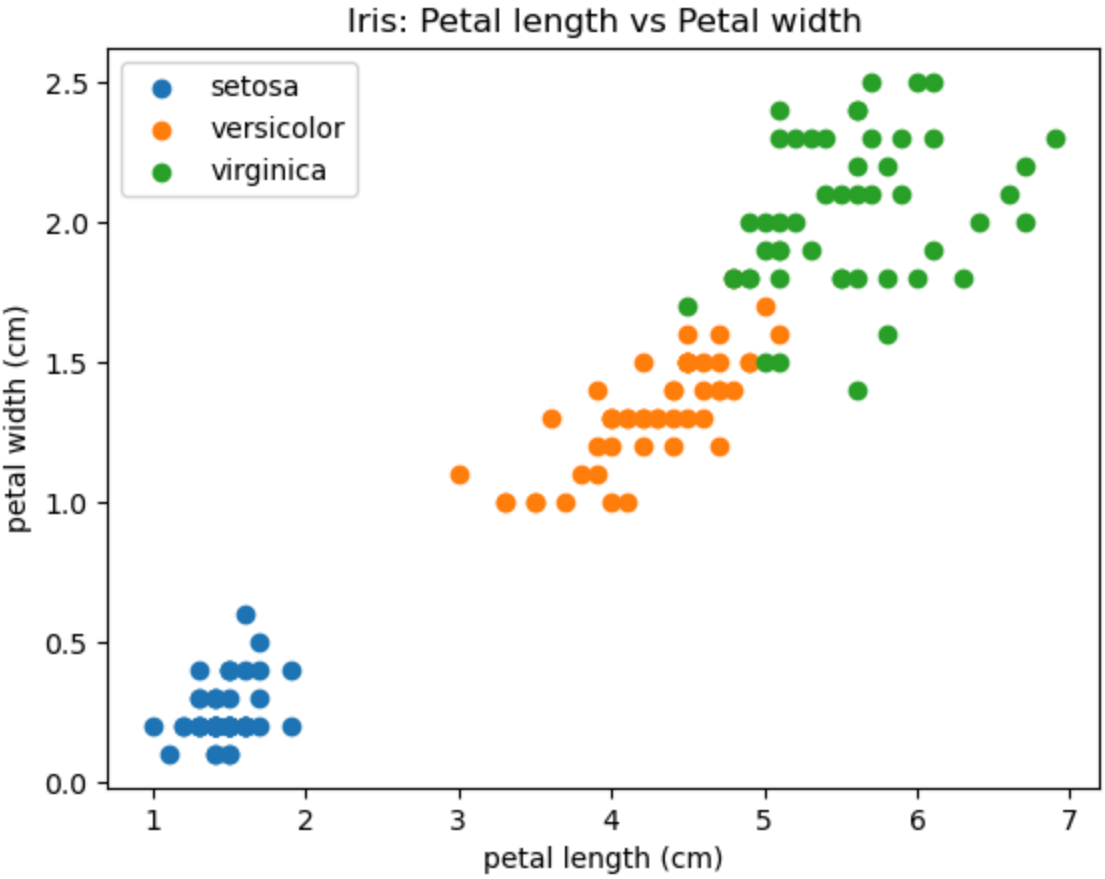
# petal length vs petal width
```

```
i,j=2,3

plt.figure()
for cls, name in enumerate(iris.target_names):
    plt.scatter(X[y==cls,i], X[y==cls,j],label=name)

plt.xlabel(iris.feature_names[i])
plt.ylabel(iris.feature_names[j])
plt.title("Iris: Petal length vs Petal width")
plt.legend()
plt.show()

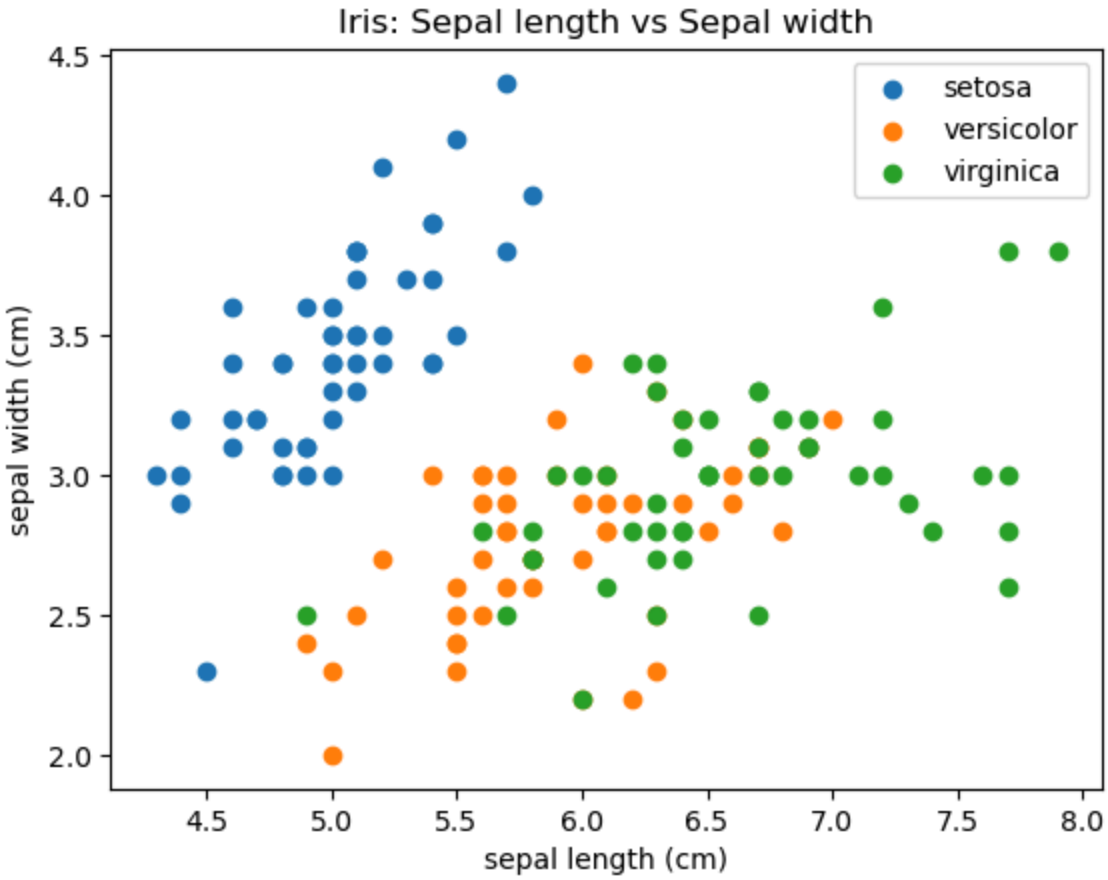
# These features are pretty good -- the separation between the three species
```



In [6]:
```
# sepal length vs sepal width
i, j = 0, 1

plt.figure()
for cls, name in enumerate(iris.target_names):
    plt.scatter(X[y==cls, i], X[y==cls, j], label=name)

plt.xlabel(iris.feature_names[i])
plt.ylabel(iris.feature_names[j])
plt.title("Iris: Sepal length vs Sepal width")
plt.legend()
plt.show()
# These features demarcate the three iris classes less cleanly !
```



Step 4: Prepare the data for ML algorithms

Machine learning algorithms often require data to be pre-processed in a certain way, such as scaling numerical values or map categories to other representations. You will also need to decide what data to use as training and as testing. You may also need to develop a strategy to deal with missing data.

Use only the first two features: **sepal length** and **width** for this assignment

Question 5

In [7]:
```
# (1 pt.) Create an array called data with the sepal length and width featur
# Create an array called targets with the target values

data=iris.data[:,0:2]
targets = iris.target


#raise NotImplementedError()
```

`sklearn` has a really helpful function called `train_test_split` that helps to randomly split your data and targets for training and testing. Please read the

documentation on `train_test_split` before running the following cell. Note that there is an important keyword `test_size`. (Look at what random_seed can do for you as well, we will use that later.)

A short summary of terminology used below:

- **training** - data used in training the machine learning model
- **test** - data used to evaluate the performance of the model. Will use this information to further refine the model.
- **validation** - the data left aside in the beginning only to be used at the very end when you are finished with your model and want to test on data the model has never seeen.

In [32]:
```
# Run this code to randomly shuffle the data and make train and validation s

x_train_all, x_validate, y_train_all, y_validate = model_selection.train_tes

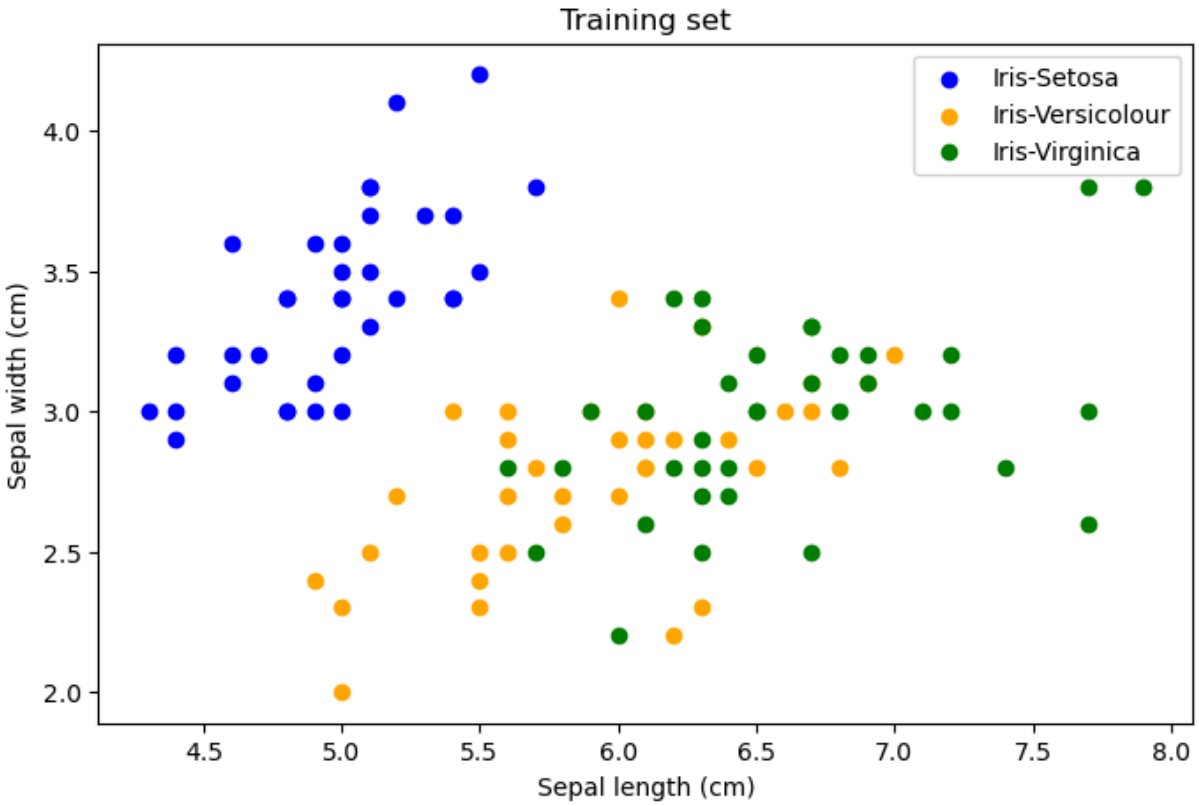
# now split the training data further into training and testing
x_train, x_test, y_train, y_test = model_selection.train_test_split(x_train_
```

In [33]:
```
# Example of plotting the training data

labels = {0: 'Iris-Setosa', 1: 'Iris-Versicolour', 2: 'Iris-Virginica'}
label_colours = ['blue', 'orange', 'green']

def plot_data(x, y, labels, colours):
    for c in np.unique(y):
        inx = np.where(y == c)
        plt.scatter(x[inx, 0], x[inx, 1], label=labels[c], c=colours[c])
    plt.title("Training set")
    plt.xlabel("Sepal length (cm)")
    plt.ylabel("Sepal width (cm)")
    plt.legend()

plt.figure(figsize=(8, 5))
plot_data(x_train, y_train, labels, label_colours)
```



Step 5: Select a model and train it

Based on the problem and the data, there are often a handful of algorithms to try. Here, experimentation and knowledge of the strengths and weaknesses of machine learning models will help you choose a model and train it.

In this lab, we will specifically use the Naive Bayes model.

Naive Bayes classifier

We will briefly review the Naive Bayes classifier model. The fundamental equation for this classifier is Bayes' rule:

$$P(Y = y_k|X_1, \ldots, X_d) = \frac{P(X_1, \ldots, X_d|Y = y_k)P(Y = y_k)}{\sum_{k=1}^{K} P(X_1, \ldots, X_d|Y = y_k)P(Y = y_k)}$$

In the above, d is the number of features or dimensions in the inputs X (in our case $d = 2$), and K is the number of classes (in our case $K = 3$). The distribution $P(Y)$ is the class prior distribution, which is a discrete distribution over K classes. The distribution $P(X|Y)$ is the class-conditional distribution over inputs.

The Naive Bayes classifier makes the assumption that the data features X_i are conditionally independent give the class Y (the 'naive' assumption). In this case, the class-conditional distribution decomposes as

$$P(X|Y=y_k) = P(X_1,\ldots,X_d|Y=y_k) \tag{1}$$
$$= \prod_{i=1}^{d} P(X_i|Y=y_k) \tag{2}$$

This simplifying assumption means that we typically need to estimate far fewer parameters for each of the distributions $P(X_i|Y=y_k)$ instead of the full joint distribution $P(X|Y=y_k)$.

Once the class prior distribution and class-conditional densities are estimated, the Naive Bayes classifier model can then make a class prediction $\hat{Y}$ for a new data input $\tilde{X} := (\tilde{X}_1,\ldots,\tilde{X}_d)$ according to

$$\hat{Y} = \text{argmax}_{y_k} P(Y=y_k|\tilde{X}_1,\ldots,\tilde{X}_d) \tag{3}$$
$$= \text{argmax}_{y_k} \frac{P(\tilde{X}_1,\ldots,\tilde{X}_d|Y=y_k)P(Y=y_k)}{\sum_{k=1}^{K} P(\tilde{X}_1,\ldots,\tilde{X}_d|Y=y_k)P(Y=y_k)} \tag{4}$$
$$= \text{argmax}_{y_k} P(\tilde{X}_1,\ldots,\tilde{X}_d|Y=y_k)P(Y=y_k) \tag{5}$$

For this lab, we will use **Gaussian distributions** to describe the distribution of sepal length and width for each of the classes of irises.

5a Compute the Priors

First, let's compute the priors for observing either three iris species. Without more information, we'll just use the fraction of each classes in the training data as the prior on that class.

In [34]:
```
class_counts =np.bincount(y_train)
priors = class_counts / len(y_train)

print("Class counts (train):", class_counts)
print("Priors P(Y=k):", priors)

for k,name in labels.items():
    print(f"P(Y={name}) = {priors[k]:.4f}")
```

```
Class counts (train): [35 36 37]
Priors P(Y=k): [0.32407407 0.33333333 0.34259259]
P(Y=Iris-Setosa) = 0.3241
P(Y=Iris-Versicolour) = 0.3333
P(Y=Iris-Virginica) = 0.3426
```

Question 6

In [35]:
```
# (1 pt.) Create an array called priors that has the prior probability of ob
# Be sure to keep track of which class belongs to which element of the array
```

```
counts=np.bincount(y_train)
priors=counts/counts.sum()

#raise NotImplementedError()
```

5b Build the Naive Bayes Classifier

Question 7

Build the Naive Bayes Classifier using Gaussian distributions. Try something

In [36]:
```
# (3 pt.) Build Classifier here

# Gaussian naive Bayes

K=len(priors)
d= x_train.shape[1]

# Estimate Gaussian parameters for each class and feature
means=np.zeros((K,d))
vars_=np.zeros((K,d))

for k in range(K):
    Xk=x_train[y_train==k]
    means[k,:] = Xk.mean(axis=0)
    vars_[k,:] = Xk.var(axis=0)
    vars_[k,:]=np.maximum(vars_[k,:],1e-9)


#raise NotImplementedError()
```

In [37]:
```
def gaussian_logpdf(x,mean,var):
    return -0.5*(np.log(2*np.pi*var) + ((x-mean)**2)/var)

def predict_gnb(X):
    log_post=np.zeros((X.shape[0],K))
    for k in range(K):
        log_prior = np.log(priors[k])
        log_lik = gaussian_logpdf(X, means[k], vars_[k]).sum(axis=1)
        log_post[:,k] = log_prior + log_lik
    return np.argmax(log_post, axis=1 )
```

In [38]:
```
y_pred = predict_gnb(x_test)

print("Test accuracy:", accuracy_score(y_test, y_pred))
```

```
Test accuracy: 0.7777777777777778
```

In [39]:
```
# Using sklearn Naive Bayes classifier
from sklearn.naive_bayes import GaussianNB

sk_gnb = GaussianNB()
```

```
sk_gnb.fit(x_train, y_train)
sk_pred = sk_gnb.predict(x_test)
print("sklearn GaussianNB accuracy:", accuracy_score(y_test, sk_pred))
```

```
sklearn GaussianNB accuracy: 0.7777777777777778
```

5c Train the model

Training the model in this case is to determine the parameters of the Gaussians describing the joint probability of observing the sepal length or width with respect to the class.

Question 8

(3 pt.) **Using as many cells as you need, determine the parameters for the joint probability distribution.**

In [40]:
```
K=3 # the three classes
d=x_train.shape[1]
print(d)
```

```
2
```

In [41]:
```
means=np.zeros((K,d))
covs=np.zeros((K,d,d))
vars_diag=np.zeros((K,d))
```

In [42]:
```
for k in range(K):
    Xk=x_train[y_train==k]
    means[k]=Xk.mean(axis=0)

    Xc = Xk-means[k]
    covs[k]=(Xc.T@Xc) / Xk.shape[0]

    vars_diag[k] = np.diag(covs[k])
```

In [43]:
```
means_df = pd.DataFrame(means,columns=["sepal length mean","sepal width mear
display(means_df)
```

	sepal length mean	sepal width mean
setosa	4.971429	3.402857
versicolor	5.936111	2.761111
virginica	6.600000	2.983784

In [27]:
```
# Covariance Matrix
for k, name in enumerate(iris.target_names):
    print(f"\nClass : {name}")
    print("Covariance matrix: \n", covs[k])
```

```
Class : setosa
Covariance matrix:
 [[0.09484429 0.08799308]
 [0.08799308 0.14851211]]

Class : versicolor
Covariance matrix:
 [[0.31645714 0.10474286]
 [0.10474286 0.09662041]]

Class : virginica
Covariance matrix:
 [[0.39224195 0.08866535]
 [0.08866535 0.10036818]]
```

5d Test the model predictions

`sklearn` comes with a lot of useful routines to compute metrics for machine learning applications. In this lab, you'll use `accuracy_score` from `sklearn.metrics` to compute the model accuracy.

Question 9

In [44]:
```
# (1 pt.) make a array called pred that is your prediction for most likely c
pred = predict_gnb(x_test)


# raise NotImplementedError()
```

In [45]:
```
# this line will give you your accuracy score for the test data
score = accuracy_score(y_test, pred)
print(score)
```

```
0.7777777777777778
```

Question 10

For this case, we will not modify our model, so we can just go ahead evaluate the accuracy of the validation data that you saved earlier.

Below, find the accuracy of the validation data.

In [46]:
```
# (1pt.) determine the accuracy score of your validation data
pred_validate = predict_gnb(x_validate)
val_accuracy = accuracy_score(y_validate, pred_validate)
val_accuracy
```

Out[46]: 0.8666666666666667

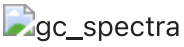
Congratulations! You have now completed the iris classification. In this iris classification, we won't go through Steps 6 to 8 of the machine learning workflow, but we will in the next part of this lab.

Classifying Stars at the Galactic Center

The Galactic center contains a complex population of stars. Multiple generations of star formation have occured, including as recently as 4 to 6 million years ago. The most reliable way to determine the ages of the stars is measure their spectra to examine their spectral features. Generally stars near the supermassive black hole in Galactic center are old (> 1 billion years) or young (4-6 million years). The stars that are bright enough for us to see are young stars and massive (> 10 Msun) with high surface temperatures (>20,000K), or old stars in their red giant phase with lower mass (~1 Msun) and cool temperatures (3000 to 4000 K). This difference in temperature leads to very different spectral features.

In the spectra below, there are two examples of young stars and one example of an old star. The young star spectra are classified as 'early-type'. Because the temperature is high, many electrons have been ionized from atoms in its atmosphere so few absorption lines remain. The most prominent absorption line for early-type stars in this wavelength region is from hydrogen (specifically, the Brackett gamma line). Sometimes, the temperature is so high that even the electron in hydrogen is ionized, so there is very little hydrogen absorption. For the old red-giant stars, the temperatures are fairly low, so there are now many atomic absorption lines. The strongest lines in this wavelength range are from sodium (Na) atoms. In addition, because the temperature is cool, the hydrogen lines become less prominant because there is not enough high energy photons that are available to make those absorption lines.

More information about the data can be found in Do et al. 2009 and Do et al. 2013 .



Feature Engineering: Equivalent Widths and Cross-Correlation

To simplify and automate the process of classifying stars, we can reduce the spectrum (flux measurements at different wavelengths) into a more compact representation that hopefully still carries a lot of the information necessary for classification. In this case, we pick the strongest spectral lines (Br gamma and Na) and turn them into features for our machine learning models. We will use the concept of equilvalent width, which is related to the depth of the absorption features. In the figure above, the shaded blue regions represent the equivalent widths of the lines. The stronger the lines, the large the

equivalent widths. For more details see: https://en.wikipedia.org/wiki/Equivalent_width. We will define positive equivalent widths as absorption lines (like in the figure above), and negative equivalent widths as emission lines.

We will also use the cross-correlation coefficient of the spectra with templates as a potential feature. This feature is the maximum from cross-correlating a template young star and old star spectrum with the observed spectrum. You can think of this correlation coefficient as how closely matched the observed spectrum is with a template spectrum. The correlation in principle should go from 0 to 1. With 0 meaning it is not at all alike and 1 being an identical match. The correlation coefficent given in the data for this lab varies widely because the observed spectra often have more noise the the spectrum above and we only cross correlate the observed spectra with one model young star and one model old star. There are also intrinsic variations in the spectra that can also reduce the correlations.

The dataset you'll be using has the equivalent widths and cross correlation coefficient measured using an automated method for spectra that have first been classified by humans, so there are labels.

Step 1: Look at the big picture

What is the problem you want to solve? How do you plan to use and benefit from the the machine learning model? What is your metric for success?

In this lab, we will give you the goal and the metric

Lab goal: classify stars at the Galactic center into young or old.

Metric: accuracy - defined as the number of correctly typed stars divided by the total number of stars

Step 2: Get the data

What data are available to you for training your model? How will you access and download all the data? The type and quantity of data available will often impact your choice in the subsequent stages.

Here, the data is straightforward. We have provided you with the training data in the file `galactic_center_stars_training.csv`

In the training data, there should be 8 columns: Na equivalent width, Na equivalent width error, correlation coefficient with an old star template, Br gamma equivalent width, Br gamma equivalent width uncertainty, signal-to-noise ratio of the spectrum, and young or

old. The units of equilvalent widths and their uncertainties are in Angstroms. The correlation coefficients are unit-less.

There is also a file called `galactic_center_stars_eval.csv` that has no labels. We'll be using that file to grade this assignment.

Question 11

In [47]:
```
# (1 pt.) In this cell, load the training data and the data for grading into
# and check that they are there by looking at each head.

train_df = pd.read_csv("galactic_center_stars_training.csv")
eval_df  = pd.read_csv("galactic_center_stars_eval.csv")

print("Training data : ")
display(train_df.head())

print("Evaluation data : ")
display(eval_df.head())


# raise NotImplementedError()
```

```
Training data :
```

	Na	Na_err	Corr	Br	Br_err	Yng_corr	SNR	Type
0	1.11	0.91	0.81	2.57	0.53	0.54	17.69	old
1	5.40	0.78	0.77	0.98	0.26	0.36	20.74	old
2	5.71	0.80	0.81	0.96	0.22	0.34	20.23	old
3	2.54	1.45	0.93	2.94	0.87	0.11	10.92	old
4	7.13	1.42	0.83	1.33	0.46	6.28	11.13	old

```
Evaluation data :
```

	Na	Na_err	Corr	Br	Br_err	Yng_corr	SNR
0	5.30	0.67	0.89	0.45	0.13	0.29	24.41
1	5.53	1.73	0.85	1.00	0.45	31.12	9.06
2	0.75	1.90	0.53	-1.52	1.20	24.63	8.20
3	7.32	2.34	0.72	2.61	1.34	0.19	6.60
4	2.91	0.80	0.88	1.10	0.27	0.23	20.12

Step 3: Explore the data

Discover and visualize the data to gain insight. For example, how complete is the dataset? Are there more representatives of certain types of data than others?

Question 12

(6 pts)

For this lab, please answer the following questions about your data. Use as many cells and plots as you like.

1. What are the features? What is the label?
2. Compute some summary statistics about your dataset. For example, the number of samples, mean, median, dispersion of the features.
3. Plot some of the features to get a sense of the data.

In [47]:
```
# use as many cells as you need to explore the data. We will be grading manu
```

1. Features -- Na equivalent width, Na equivalent width error, correlation coefficient with an old star template, Br gamma equivalent width, Br gamma equivalent width uncertainty, signal-to-noise ratio of the spectrum.

   Label -- Young versus Old

In [48]:
```
#2) Summary statistics about dataset

eval_df = pd.read_csv("galactic_center_stars_eval.csv")
label_col = [c for c in train_df.columns if c not in eval_df.columns][0]
feature_cols = [c for c in train_df.columns if c != label_col]

print("N samples:", len(train_df))
print("\nClass counts:")
print(train_df[label_col].value_counts())

print("\nSummary stats (mean, std, median, etc.):")
display(train_df[feature_cols].describe())

print("\nMedians:")
display(train_df[feature_cols].median())
```

```
N samples: 419

Class counts:
Type
old      299
young    120
Name: count, dtype: int64

Summary stats (mean, std, median, etc.):
```

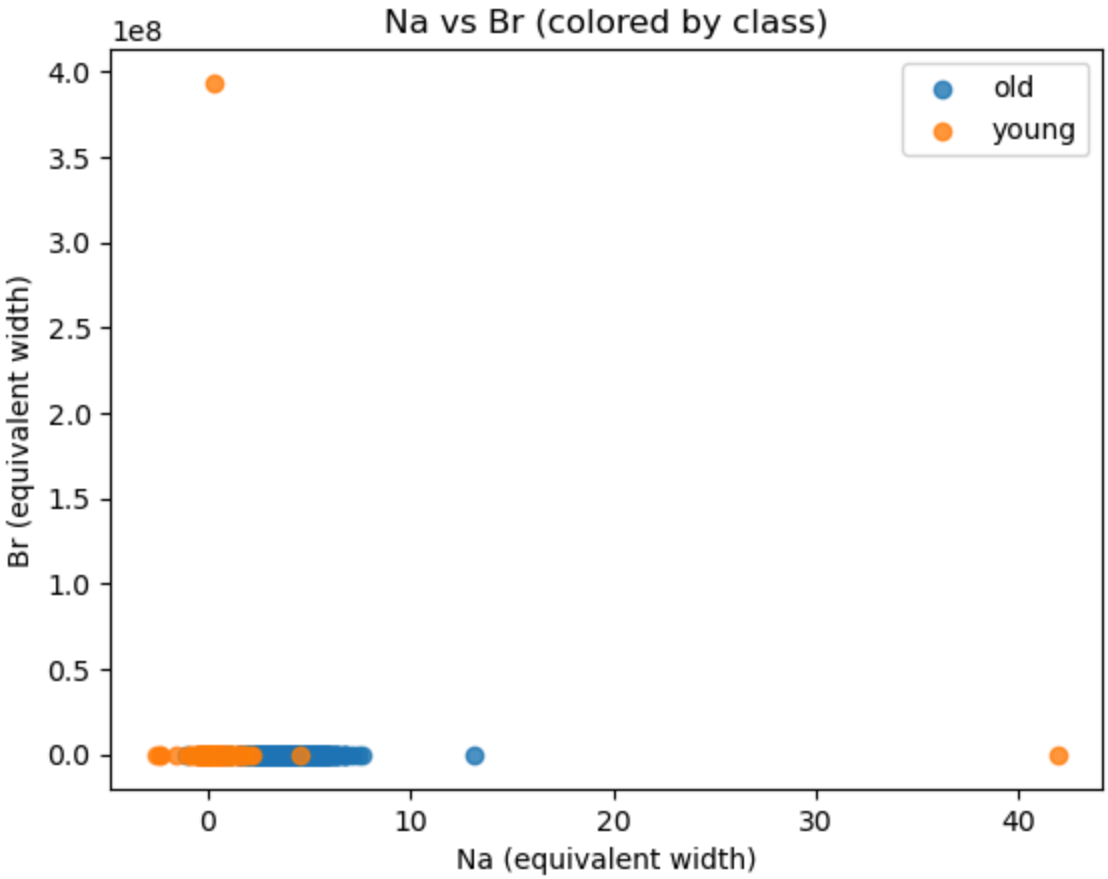
	Na	Na_err	Corr	Br	Br_err	Yng_c
count	419.000000	417.000000	419.000000	4.190000e+02	4.190000e+02	4.190000e+
mean	3.114630	1.341487	6.307780	9.379330e+05	8.194493e+08	8.085288e+
std	2.893386	6.151558	42.377727	1.921929e+07	1.642226e+10	1.141227e+
min	-2.560000	0.240000	-0.880000	-4.318304e+05	0.000000e+00	-2.550000e+
25%	1.020000	0.680000	0.560000	1.015000e+00	2.700000e-01	3.000000e-
50%	3.440000	0.900000	0.700000	2.050000e+00	4.900000e-01	4.500000e-
75%	4.670000	1.130000	0.830000	3.765000e+00	8.400000e-01	7.650000e-
max	41.940000	125.390000	740.740000	3.934078e+08	3.360945e+11	1.795982e+

```
Medians:
Na           3.44
Na_err       0.90
Corr         0.70
Br           2.05
Br_err       0.49
Yng_corr     0.45
SNR         17.78
dtype: float64
```

In [49]:
```python
# 3) Plotting some features

plt.figure()
for cls in train_df[label_col].unique():
    sub = train_df[train_df[label_col] == cls]
    plt.scatter(sub["Na"], sub["Br"], label=str(cls), alpha=0.8)

plt.xlabel("Na (equivalent width)")
plt.ylabel("Br (equivalent width)")
plt.title("Na vs Br (colored by class)")
plt.legend()
plt.show()
```



In [50]:
```
# feature columns
feature_cols = ["Na","Na_err","Corr","Br","Br_err","Yng_corr","SNR"]

# From the scatter plot above, it is clear that the data has some clear outl

# Preliminary Cleaning by imposing quartiles

# 1) start clean: drop missing + obvious bad rows
clean_training = train_df.dropna(subset=feature_cols + [label_col]).copy()
clean_training = clean_training[
    (clean_training["Na_err"] >= 0) &
    (clean_training["Br_err"] >= 0) &
    (clean_training["SNR"] > 0)
].copy()

# 2) remove outliers via IQR rule (flag if ANY feature is outside bounds)
outlier_mask = pd.Series(False, index=clean_training.index)

for col in feature_cols:
    q1 = clean_training[col].quantile(0.25)
    q3 = clean_training[col].quantile(0.75)
    iqr = q3 - q1
    low = q1 - 1.5 * iqr
    high = q3 + 1.5 * iqr
    outlier_mask |= (clean_training[col] < low) | (clean_training[col] > hig

print("Rows before:", len(train_df))
print("After drop bad/missing:", len(clean_training))
```

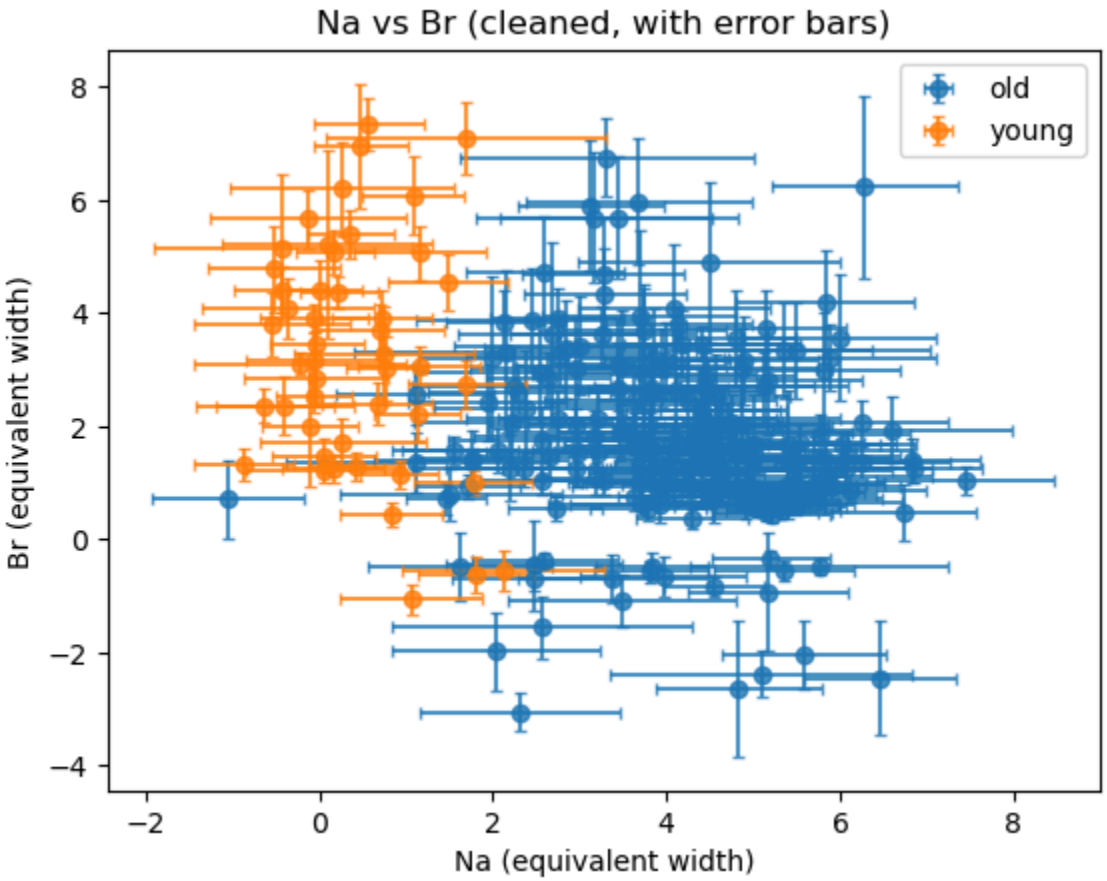
```
print("Outliers removed:", int(outlier_mask.sum()))

clean_training = clean_training.loc[~outlier_mask].copy()
print("Rows after outlier removal:", len(clean_training))
```

```
Rows before: 419
After drop bad/missing: 417
Outliers removed: 160
Rows after outlier removal: 257
```

In [51]:
```
plt.figure()
for cls in clean_training[label_col].unique():
    sub = clean_training[clean_training[label_col] == cls]
    plt.errorbar(
        sub["Na"], sub["Br"],
        xerr=sub["Na_err"], yerr=sub["Br_err"],
        fmt="o", alpha=0.8, capsize=2,
        label=str(cls)
    )

plt.xlabel("Na (equivalent width)")
plt.ylabel("Br (equivalent width)")
plt.title("Na vs Br (cleaned, with error bars)")
plt.legend()
plt.show()
```



Step 4: Prepare the data for ML algorithms

Machine learning algorithms often require data to be pre-processed in a certain way, such as scaling numerical values or map categories to other representations. You will also need to decide what data to use as training and as testing. You may also need to develop a strategy to deal with missing data.

For this lab, we have already done the feature engineering to get the equivalent widths, their uncertainties, and the correlation coefficients.

Question 12

(6 pts)

While the features are already chosen, there are a number of other questions that need to be answered:

1. Are there bad data points or outliers?
2. If there are outliers, what is your strategy for dealing with them?
3. Apply your strategy for cleaning and dealing with the data, then split your training data into three parts: training, testing, and validation

In [ ]:
```
# use as many cells as you need two answer Question 12
```

In [61]:
```
# Describe your process, clean the data, and show clean_training.describe()
```

1. Yes there are bad data points.
2. Outliers removed by putting hard bounds on the data -- for instance, the correlation should be between 0 and 1. Any data points outside these have been removed. There are two rows with NaN entries - they have been removed as well.

In [87]:
```
#Systematically cleaning the data by imposing hard bounds on the data !

def remove_outliers_by_bounds(
    df: pd.DataFrame,
    bounds: dict,
    *,
    include_nan_as_outlier: bool = False,
    keep_cols: list[str] | None = None,
    verbose: bool = True,
) -> pd.DataFrame:

    if keep_cols is not None:
        bounds = {c: b for c, b in bounds.items() if c in keep_cols}

    missing = [c for c in bounds.keys() if c not in df.columns]
    if missing:
        raise KeyError(f"These columns are not in df: {missing}")
```

```
    mask_keep = pd.Series(True, index=df.index)

    for col, (low, high) in bounds.items():
        s = df[col]

        if include_nan_as_outlier:
            ok = ~s.isna()
        else:
            ok = pd.Series(True, index=df.index)

        if low is not None:
            ok &= (s >= low) | (s.isna() if not include_nan_as_outlier else

        if high is not None:
            ok &= (s <= high) | (s.isna() if not include_nan_as_outlier else

        mask_keep &= ok

    removed = int((~mask_keep).sum())
    if verbose:
        print(f"Removed {removed} rows out of {len(df)} ({removed/len(df):.1

    return df.loc[mask_keep].copy()
```

In [88]:
```
bounds = {
    "Na": (-1, 7),
    "Br": (0, 10),
    "Corr": (0, 1),
    "Yng_corr": (0, 1)
    #"Na_err": (0, 0.9)   # adjust if this feature can be negative
}

clean_training = remove_outliers_by_bounds(train_df, bounds)
```

```
Removed 157 rows out of 419 (37.5%).
```

In [89]:
```
feature_cols = ["Na","Na_err","Corr","Br","Br_err","Yng_corr","SNR"]

print("N rows:", len(clean_training))
display(clean_training[feature_cols].describe())
```

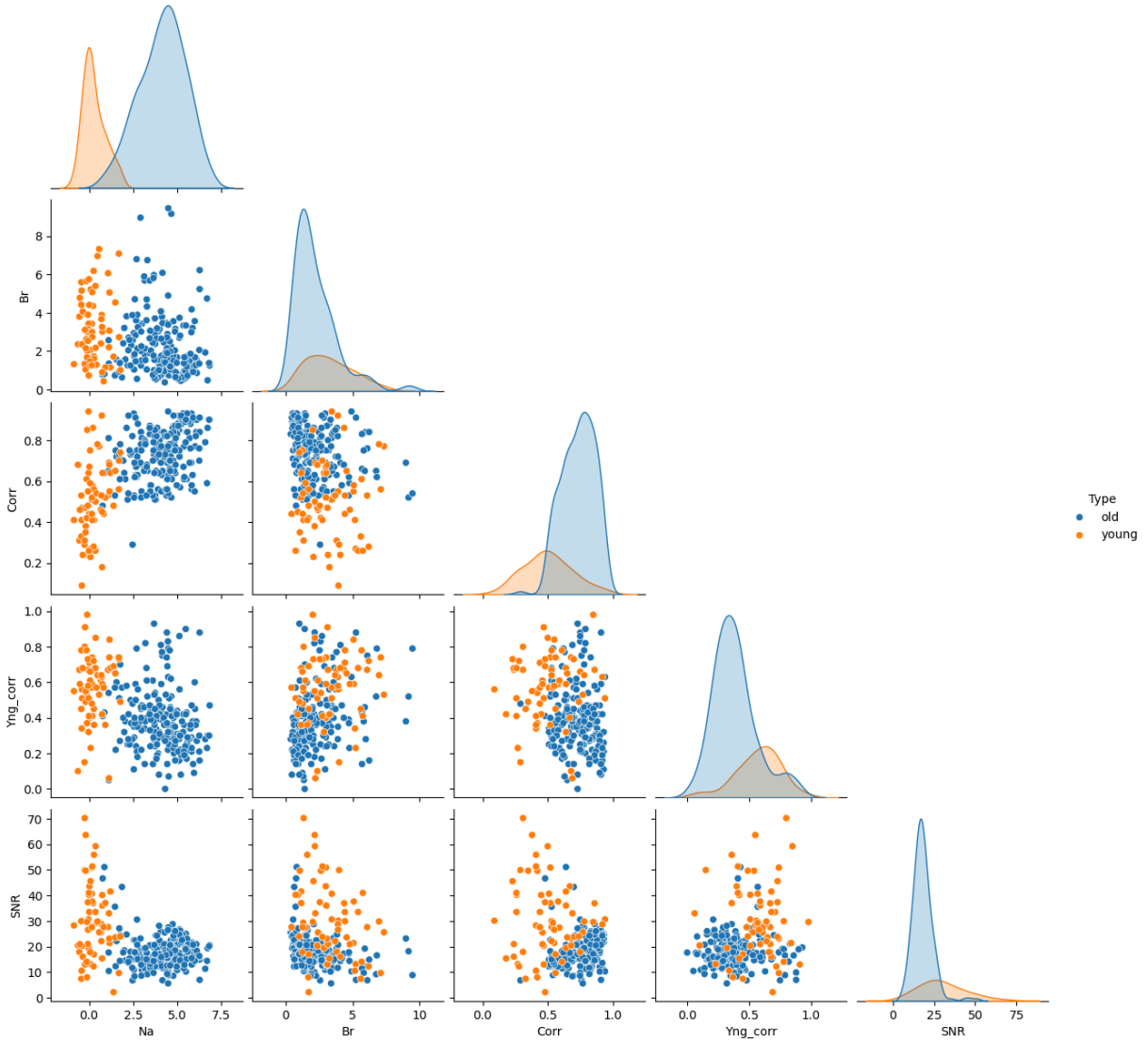
```
N rows: 262
```

	Na	Na_err	Corr	Br	Br_err	Yng_corr	
count	262.000000	262.000000	262.000000	262.000000	262.000000	262.000000	262
mean	3.200344	0.944962	0.676031	2.462328	0.554084	0.429504	20
std	2.041349	0.520799	0.172878	1.708151	0.412991	0.196843	10
min	-0.860000	0.240000	0.090000	0.370000	0.140000	-0.000000	2
25%	1.495000	0.680000	0.560000	1.232500	0.260000	0.290000	14
50%	3.710000	0.885000	0.700000	2.000000	0.455000	0.400000	18
75%	4.797500	1.070000	0.817500	3.240000	0.697500	0.560000	23
max	6.850000	6.750000	0.940000	9.460000	3.110000	0.980000	70

In [98]:
```
# Before splitting the training data into trainig, testing, and validation s
# plot clean training with sns.pairplot(clean_training, hue="Type")
```

In [55]:
```
# Plotting the cleaned and processed data !

sns.pairplot(
    clean_training[["Na", "Br", "Corr", "Yng_corr", "SNR", label_col]],
    hue=label_col,
    corner=True
)
plt.show()
```



In [150…

```
# After cleaning, split data into training, testing, and validation

# YOUR CODE HERE
#raise NotImplementedError()
feature_cols = ["Na","Na_err","Corr","Br","Br_err","Yng_corr","SNR"]
data = clean_training[feature_cols].to_numpy()
targets = clean_training[label_col].to_numpy()

# Run this code to randomly shuffle the data and make train and validation s
x_train_all, x_validate, y_train_all, y_validate = model_selection.train_tes
    data, targets, test_size=0.1, stratify=targets
)

# now split the training data further into training and testing
x_train, x_test, y_train, y_test = model_selection.train_test_split(
    x_train_all, y_train_all, test_size=0.2, stratify=y_train_all
)
```

Step 5: Select a model and train it

Based on the problem and the data, there are often a handful of algorithms to try. Here, experimentation and knowledge of the strengths and weaknesses of machine learning models will help you choose a model and train it.

Question 13

(10 pts)

In this lab, we will specifically use the Naive Bayes model.

In the following cells, please implement the Naive Bayes classifier to deterimine whether a star is young or old based on the features you've selected in Step 4.
At the end of this step, report the accuracy of your model on the test data.

In [ ]:
```
# use as many cells as you need to do Step 5. We will be grading manually
```

In [ ]:
```
# Uncomment these and run when ready
#from sklearn.metrics import accuracy_score
#accuracy_score(y_test, pred)
```

In [147…
```
# nb_accuracy takes in as inputs any choice of features we choose to include

def nb_accuracy(x_train, y_train, x_eval, y_eval, feature_idx=None, var_floo
    """
    Train Gaussian Naive Bayes on (x_train, y_train),
    then predict on (x_eval, y_eval) using selected features,
    and return accuracy.

    feature_idx: list of column indices to use (e.g. [0,3,6]).
                 If None, uses all features.
    """
    # select features
    Xtr = x_train if feature_idx is None else x_train[:, feature_idx]
    Xev = x_eval  if feature_idx is None else x_eval[:, feature_idx]

    # encode labels (works for strings or ints)
    classes, ytr_enc = np.unique(y_train, return_inverse=True)
    class_to_int = {c:i for i, c in enumerate(classes)}
    yev_enc = np.array([class_to_int[c] for c in y_eval])

    K = len(classes)
    d = Xtr.shape[1]

    # priors
    counts = np.bincount(ytr_enc, minlength=K)
    priors = counts / counts.sum()

    # Gaussian params
    means = np.zeros((K, d))
    vars_ = np.zeros((K, d))
    for k in range(K):
        Xk = Xtr[ytr_enc == k]
```

```
        means[k] = Xk.mean(axis=0)
        vars_[k]  = Xk.var(axis=0)          # ddof=0
        vars_[k]  = np.maximum(vars_[k], var_floor)

    # predict
    log_post = np.zeros((Xev.shape[0], K))
    for k in range(K):
        log_prior = np.log(priors[k])
        log_lik = -0.5 * (np.log(2*np.pi*vars_[k]) + ((Xev - means[k])**2)/v
        log_post[:, k] = log_prior + log_lik

    pred_enc = np.argmax(log_post, axis=1)

    # accuracy
    return accuracy_score(yev_enc, pred_enc)
```

In [151…
```
# Accuracy on training set
acc_train = nb_accuracy(x_train, y_train, x_train, y_train, feature_idx=[0,3
print(acc_train)
```

```
0.9787234042553191
```

In [152…
```
#Accuracy on test set
acc_test = nb_accuracy(x_train, y_train, x_test, y_test, feature_idx=[0,3])
print(acc_test)
```

```
0.9574468085106383
```

Accuracy on the test set :

$$95.7\%$$

Step 6: Fine tune the model

Most machine learning algorithms will have hyperparameters that can be tuned to improve the performance of the model given the specific dataset. You may also want to visualize the results of model predictions to aid in tuning the model.

Question 14

(9 pts)

For this lab, please do the following:

1. One of the hyperparameters in this case is the types of features to use. Try subtracting or adding a feature to see how it changes your accuracy.
2. Examine how changes in your training data might affect your results. For example, what if you use only measurements with small uncertainties for training?
3. Report your final accuracy on your test dataset after selecting the optimal model. Explain why you think this is the optimal model.

In [ ]:
```
# use as many cells as you need to do Step 6. We will be grading manually
```

In [94]:
```
# ALL FEATURES considered together
# Accuracy on train data
acc_train = nb_accuracy(x_train, y_train, x_train, y_train, feature_idx=None
print(acc_train)
#Accuracy on test set
acc_test = nb_accuracy(x_train, y_train, x_test, y_test, feature_idx=None)
print(acc_test)
```

```
0.973404255319149
0.9574468085106383
```

Accuracy of the model when all the features are considered : 97.3% on the training set

Accuracy of the model when all the features are considered : 95.7% on the test set

In [98]:
```
# Na, Br and Yng_Corr considered together
# Accuracy on train data
acc_train = nb_accuracy(x_train, y_train, x_train, y_train, feature_idx=[0,3
print(acc_train)
#Accuracy on test set
acc_test = nb_accuracy(x_train, y_train, x_test, y_test, feature_idx=[0,3])
print(acc_test)
```

```
0.9680851063829787
0.9574468085106383
```

Accuracy of the model when only Na and Br features are considered : 96.8% on the training set

Accuracy of the model when only Na and Br features are considered : 95.7% on the test set

In [66]:
```
#2) Use data with ONLY small uncertainities -- restric the data

bounds = {
    "Na": (-1, 7),
    "Br": (0, 10),
    "Corr": (0, 1),
    "Yng_corr": (0, 1),
    "Na_err": (0, 0.9)   # Adjust the uncertainities here !
}

clean_training = remove_outliers_by_bounds(train_df, bounds)

# We remove 281 rows now !
```

```
Removed 281 rows out of 419 (67.1%).
```

In [85]:
```
# This is the new split after including just the data with small uncertainit
# After cleaning, split data into training, testing, and validation

# YOUR CODE HERE
```

```
#raise NotImplementedError()
feature_cols = ["Na","Na_err","Corr","Br","Br_err","Yng_corr","SNR"]
data = clean_training[feature_cols].to_numpy()
targets = clean_training[label_col].to_numpy()

# Run this code to randomly shuffle the data and make train and validation s
x_train_all, x_validate, y_train_all, y_validate = model_selection.train_tes
    data, targets, test_size=0.1, stratify=targets
)

# now split the training data further into training and testing
x_train, x_test, y_train, y_test = model_selection.train_test_split(
    x_train_all, y_train_all, test_size=0.2, stratify=y_train_all
)
```

In [86]:
```
# ALL FEATURES considered together
# Accuracy on train data
acc_train = nb_accuracy(x_train, y_train, x_train, y_train, feature_idx=None
print(acc_train)
#Accuracy on test set
acc_test = nb_accuracy(x_train, y_train, x_test, y_test, feature_idx=None)
print(acc_test)
```

```
0.9696969696969697
0.96
```

By restricting to data with low uncertainities, we have data that is confined to high quality measurements, so the classes become much easier to separate and the model generalizes very well inside this filtered regime. Low Na_er selects higher quality spectra, so the measured Na/Br features are more reliable. This reduces the overlap between young/old in the feature space and hence Naive Bayes works better. The tradeoff is that we classify fewer start and the model may underperform on noisier spectra.

In [153…
```
import itertools

def eval_feature_subsets(
    x_train, y_train, x_test, y_test,
    feature_names=None,
    min_features=1,
    only_red_flags=True,
    min_gap=0.0,          # e.g. 0.02 to only flag gaps >= 2%
    sort_by_gap=True
):
    """
    Evaluate feature subsets using nb_accuracy(..., feature_idx=subset).

    If only_red_flags=True:
      keeps only subsets with (train_acc - test_acc) >= min_gap
      (default min_gap=0.0 means test < train)
    """
    d = x_train.shape[1]
    if feature_names is None:
        feature_names = [f"f{i}" for i in range(d)]
```

```
    rows = []
    all_idx = list(range(d))

    for k in range(min_features, d + 1):
        for subset in itertools.combinations(all_idx, k):
            subset = list(subset)

            acc_train = nb_accuracy(x_train, y_train, x_train, y_train, feat
            acc_test  = nb_accuracy(x_train, y_train, x_test,  y_test,  feat
            gap = acc_train - acc_test

            if only_red_flags and not (gap >= min_gap and acc_test < acc_tra
                continue

            rows.append({
                "k": k,
                "feature_idx": subset,
                "features": [feature_names[i] for i in subset],
                "train_acc": acc_train,
                "test_acc": acc_test,
                "gap": gap
            })

    df = pd.DataFrame(rows)
    if len(df) == 0:
        return df

    if sort_by_gap:
        df = df.sort_values(["gap", "test_acc"], ascending=[False, False])
    else:
        df = df.sort_values(["test_acc", "train_acc", "k"], ascending=[False

    return df.reset_index(drop=True)
```

In [156…

```
results = eval_feature_subsets(x_train, y_train, x_test, y_test,
                               feature_names=feature_names, min_features=1,
                               only_red_flags=False)   # make sure it's NOT

thr = 0.95

good = results[(results.train_acc >= thr) & (results.test_acc >= thr)].copy(
good["gap"] = good["train_acc"] - good["test_acc"]
good["min_acc"] = good[["train_acc","test_acc"]].min(axis=1)

good.sort_values(["min_acc", "gap", "k"], ascending=[False, True, True]).hea
```

Out[156…

	k	feature_idx	features	train_acc	test_acc	gap	min_acc
125	4	[0, 2, 4, 6]	[Na, Corr, Br_err, SNR]	0.968085	0.957447	0.010638	0.957447
122	1	[0]	[Na]	0.973404	0.957447	0.015957	0.957447
123	2	[0, 2]	[Na, Corr]	0.973404	0.957447	0.015957	0.957447
118	2	[0, 3]	[Na, Br]	0.978723	0.957447	0.021277	0.957447

Selected the feature subset maximising the minimum of train / test accuracy (by first ensuring that both are high with threshold being 95%). Amongst these, we choose the smallest test-train gap in accuracy !

For example, models with test accuracy 100% are probably too tuned, so remove them from consideration. An optimal model one seems to be where we consider feature combinations [Na, Br] with train accuracy 97.8% and test accuracy of 95.7%. These are pretty close to each other and with more examples of splits most viable to be stable and close to each other !

Step 7: Present your solution and apply it

Here you will evaluate your model against the test set that was set aside in Step 2 to determine the final performance metrics. Your model is now ready to be applied to new data.

Question 15

(2 pts)

For this lab, please do the following:

1. Report your accuracy for the validation data that you saved in Step 4. How does this score compare to when you were training and fine tuning your model?
2. Apply your model to the data for the unlabeled grading data. Note that you should explore these inputs to see if there are differences in the features between your training data and this dataset. Beware of outliers in this dataset as well. Create a set of predictions called pred_eval.

In [157…

```
# use as many cells as you need to do Step 7.
acc_validate = nb_accuracy(x_train, y_train, x_validate, y_validate, feature
print(acc_validate)
```

```
0.9629629629629629
```

Accuracy for the validation data : 96.29%

If you would like to submit your group's solution to the unlabeled dataset for the competition to see who gets the accurate predictions, please send your TA and Prof. Do a csv file with a single column predicting whether each star is 'young' or 'old'. Below is an example code of how to create this file. You will send ONE file per team.

Done and Submitted to Prof. Do !

In [ ]:
```
import pandas as pd

# if your answer for three stars were:
answer = ['old','young','old']

# change the list into a data frame
answer_tab = pd.DataFrame({'target':answer})

# save the data frame into a csv file
answer_tab.to_csv('group_x_answer.csv',index=None)
```